

Employee Churn Model with a Strategic Retention Plan

```
In [10]: import warnings
warnings.filterwarnings('ignore')
```

```
In [11]: import pandas as pd
from pandas.plotting import scatter_matrix
from pandas import ExcelWriter
from pandas import ExcelFile
from openpyxl import load_workbook
import numpy as np
from scipy.stats import norm, skew
from scipy import stats
import statsmodels.api as sm
```

```
In [12]: # importing libraries for data visualisations
import seaborn as sns
from matplotlib import pyplot
import matplotlib.pyplot as plt
import matplotlib.pylab as pylab
import matplotlib
%matplotlib inline
color = sns.color_palette()
from IPython.display import display
pd.options.display.max_columns = None
# Standard plotly imports
from chart_studio import plotly as py
import plotly as py
import plotly.figure_factory as ff
import plotly.graph_objs as go
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
init_notebook_mode(connected=True)
#py.init_notebookmode(connected=True) # this code, allow us to work with offline
# Using plotly + cufflinks in offline mode
import cufflinks as cf
cf.set_config_file(offline=True)
import cufflinks
cufflinks.go_offline(connected=True)
```

```
In [13]: # sklearn modules for preprocessing
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
# from imblearn.over_sampling import SMOTE # SMOTE
# sklearn modules for ML model selection
from sklearn.model_selection import train_test_split # import 'train_test_split'
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

# Libraries for data modelling
from sklearn import svm, tree, linear_model, neighbors
from sklearn import naive_bayes, ensemble, discriminant_analysis, gaussian_process
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier

# Common sklearn Model Helpers
from sklearn import feature_selection
from sklearn import model_selection
from sklearn import metrics
# from sklearn.datasets import make_classification

# sklearn modules for performance metrics
from sklearn.metrics import confusion_matrix, classification_report, precision_score
from sklearn.metrics import auc, roc_auc_score, roc_curve, recall_score, log_loss
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score, make_scorer
from sklearn.metrics import average_precision_score
```

```
In [14]: # importing miscellaneous libraries
import os
import re
import sys
import timeit
import string
from datetime import datetime
from time import time
from dateutil.parser import parse
# ip = get_ipython()
# ip.register_magics(jupyternotify.JupyterNotifyMagics)
```

```
In [15]: # Read Excel file
df_sourcefile = pd.read_csv(r"C:\Users\JANHAVI\Downloads\WA_Fn-UseC_-HR-Employee Churn.csv")
print("Shape of dataframe is: {}".format(df_sourcefile.shape))
```

Shape of dataframe is: (1470, 35)

```
In [16]: # Make a copy of the original sourcefile
df_HR = df_sourcefile.copy()
```

```
In [17]: # Dataset columns
df_HR.columns
```

```
Out[17]: Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
               'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',
               'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',
               'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
               'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
               'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
               'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',
               'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
               'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
               'YearsWithCurrManager'],
              dtype='object')
```

```
In [18]: # Dataset header
df_HR.head()
```

```
Out[18]:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educ
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Lif
1	49	No	Travel_Frequently	279	Research & Development	8	1	Lif
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Lif
4	27	No	Travel_Rarely	591	Research & Development	2	1	

```
In [19]: # Let's break down the columns by their type (i.e. int64, float64, object)
df_HR.columns.to_series().groupby(df_HR.dtypes).groups
```

```
Out[19]: {int64: ['Age', 'DailyRate', 'DistanceFromHome', 'Education', 'EmployeeCount',
               'EmployeeNumber', 'EnvironmentSatisfaction', 'HourlyRate', 'JobInvolvement',
               'JobLevel', 'JobSatisfaction', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
               'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction',
               'StandardHours', 'StockOptionLevel', 'TotalWorkingYears', 'TrainingTimesLastYear',
               'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
               'YearsWithCurrManager'], object: ['Attrition', 'BusinessTravel', 'Department',
               'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'Over18', 'OverTime']}
```

```
In [20]: # Columns datatypes and missign values
df_HR.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   1470 non-null   int64
1   Attrition                           1470 non-null   object
2   BusinessTravel                      1470 non-null   object
3   DailyRate                           1470 non-null   int64
4   Department                          1470 non-null   object
5   DistanceFromHome                   1470 non-null   int64
6   Education                           1470 non-null   int64
7   EducationField                      1470 non-null   object
8   EmployeeCount                       1470 non-null   int64
9   EmployeeNumber                     1470 non-null   int64
10  EnvironmentSatisfaction             1470 non-null   int64
11  Gender                              1470 non-null   object
12  HourlyRate                          1470 non-null   int64
13  JobInvolvement                     1470 non-null   int64
14  JobLevel                           1470 non-null   int64
15  JobRole                             1470 non-null   object
16  JobSatisfaction                     1470 non-null   int64
17  MaritalStatus                      1470 non-null   object
18  MonthlyIncome                      1470 non-null   int64
19  MonthlyRate                         1470 non-null   int64
20  NumCompaniesWorked                 1470 non-null   int64
21  Over18                             1470 non-null   object
22  OverTime                           1470 non-null   object
23  PercentSalaryHike                  1470 non-null   int64
24  PerformanceRating                  1470 non-null   int64
25  RelationshipSatisfaction            1470 non-null   int64
26  StandardHours                      1470 non-null   int64
27  StockOptionLevel                   1470 non-null   int64
28  TotalWorkingYears                  1470 non-null   int64
29  TrainingTimesLastYear              1470 non-null   int64
30  WorkLifeBalance                    1470 non-null   int64
31  YearsAtCompany                     1470 non-null   int64
32  YearsInCurrentRole                  1470 non-null   int64
33  YearsSinceLastPromotion             1470 non-null   int64
34  YearsWithCurrManager                1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

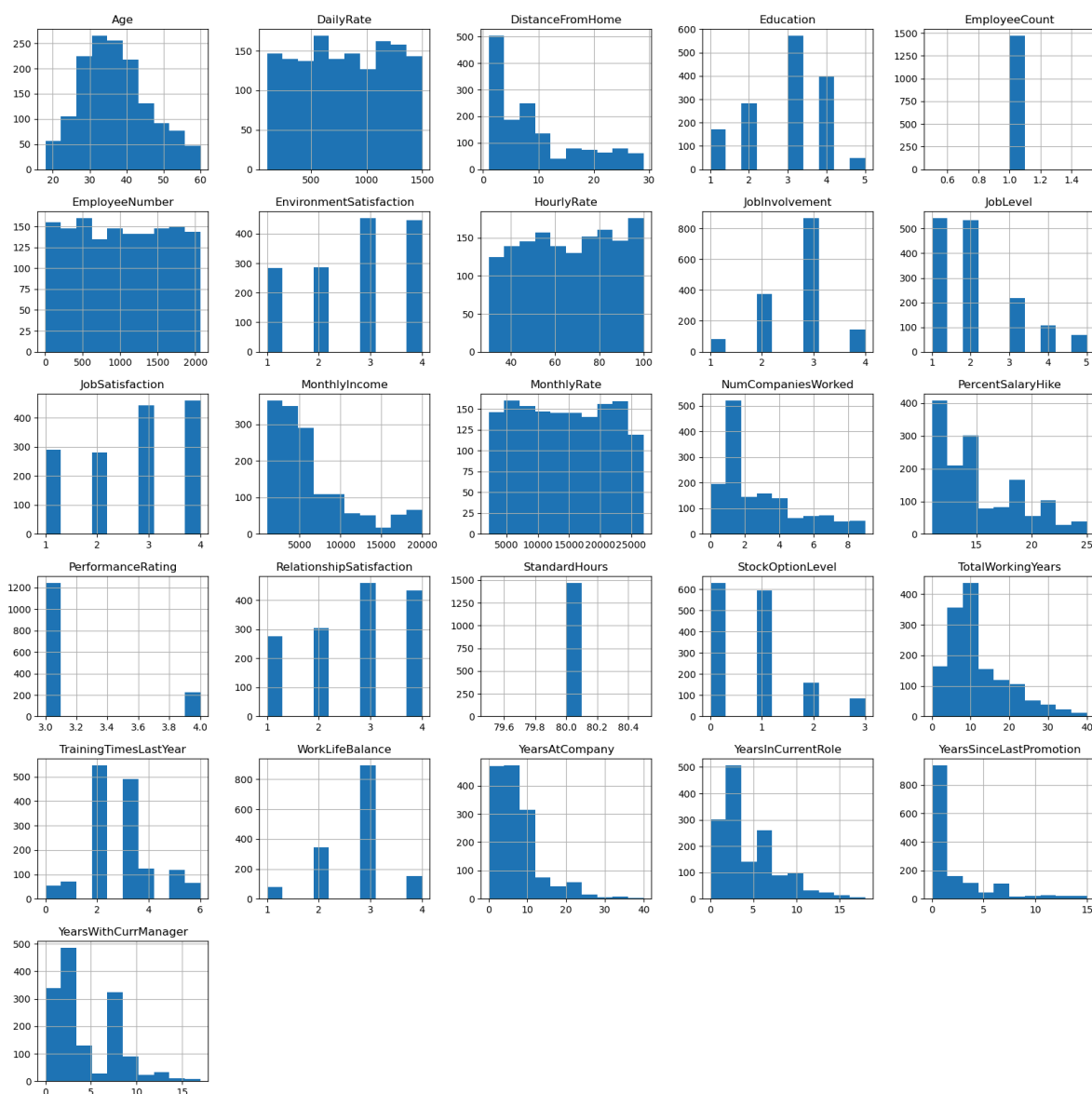


```
In [21]: df_HR.describe()
```

```
Out[21]:
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNu
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	1470.00
mean	36.923810	802.485714	9.192517	2.912925	1.0	1024.86
std	9.135373	403.509100	8.106864	1.024165	0.0	602.02
min	18.000000	102.000000	1.000000	1.000000	1.0	1.00
25%	30.000000	465.000000	2.000000	2.000000	1.0	491.25
50%	36.000000	802.000000	7.000000	3.000000	1.0	1020.50
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1555.75
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.00

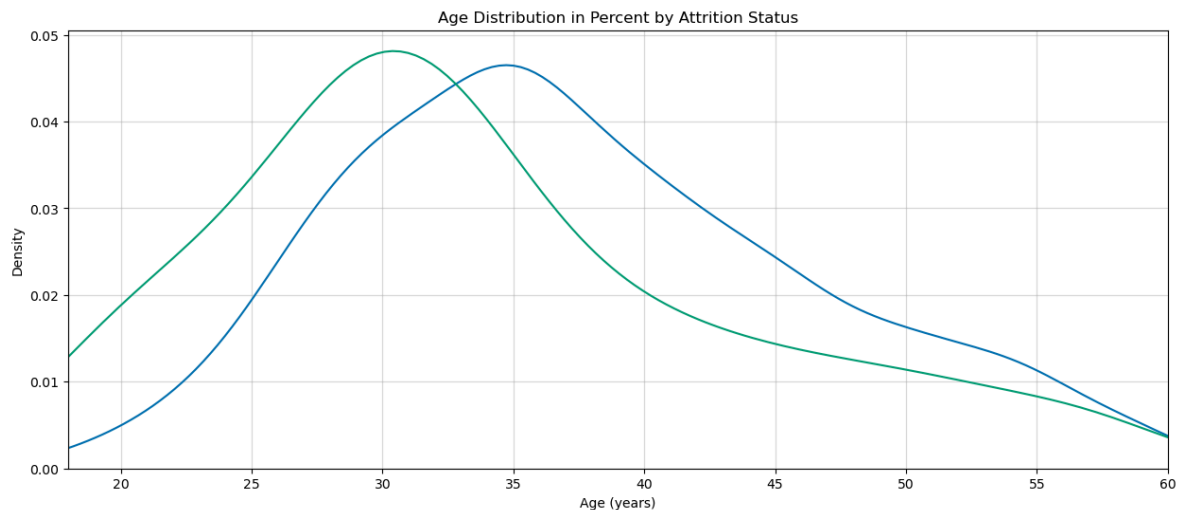
```
In [22]: df_HR.hist(figsize=(20,20))
plt.show()
```



```
In [23]: (mu, sigma) = norm.fit(df_HR.loc[df_HR['Attrition'] == 'Yes', 'Age'])
print(
    'Ex-employees: average age = {:.1f} years old and standard deviation = {:.1f}'
    .format(mu, sigma))
(mu, sigma) = norm.fit(df_HR.loc[df_HR['Attrition'] == 'No', 'Age'])
print('Current employees: average age = {:.1f} years old and standard deviation = {:.1f}'
      .format(mu, sigma))
```

Ex-employees: average age = 33.6 years old and standard deviation = 9.7
 Current employees: average age = 37.6 years old and standard deviation = 8.9

```
In [24]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'Age'], label = 'Active Employees')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'Age'], label = 'Ex-Employees')
plt.xlim(left=18, right=60)
plt.xlabel('Age (years)')
plt.ylabel('Density')
plt.title('Age Distribution in Percent by Attrition Status');
```



```
In [25]: # Education Field of employees
df_HR['EducationField'].value_counts()
```

```
Out[25]: EducationField
Life Sciences      606
Medical           464
Marketing          159
Technical Degree   132
Other              82
Human Resources    27
Name: count, dtype: int64
```

```
In [26]: df_EducationField = pd.DataFrame(columns=["Field", "% of Leavers"])
i=0
for field in list(df_HR['EducationField'].unique()):
    ratio = df_HR[(df_HR['EducationField']==field)&(df_HR['Attrition']=="Yes")]
    df_EducationField.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_EF = df_EducationField.groupby(by="Field").sum()
df_EF.iplot(kind='bar',title='Leavers by Education Field (%)')
```



```
In [27]: # Gender of employees
df_HR['Gender'].value_counts()
```

```
Out[27]: Gender
Male      882
Female    588
Name: count, dtype: int64
```

```
In [28]: print("Normalised gender distribution of ex-employees in the dataset: Male = {
    df_HR['Gender'] == 'Male'}).shape[0] / df_HR[df_HR['Gender'] == 'Male'].sh
```

```
Normalised gender distribution of ex-employees in the dataset: Male = 17.0%;
Female 14.8%.
```

```
In [29]: df_Gender = pd.DataFrame(columns=["Gender", "% of Leavers"])
i=0
for field in list(df_HR['Gender'].unique()):
    ratio = df_HR[(df_HR['Gender']==field)&(df_HR['Attrition']=="Yes")].shape[0]
    df_Gender.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_G = df_Gender.groupby(by="Gender").sum()
df_G.iplot(kind='bar',title='Leavers by Gender (%)')
```



```
In [30]: # Marital Status of employees
df_HR['MaritalStatus'].value_counts()
```

```
Out[30]: MaritalStatus
Married      673
Single       470
Divorced     327
Name: count, dtype: int64
```



```
In [31]: df_Marital = pd.DataFrame(columns=["Marital Status", "% of Leavers"])
i=0
for field in list(df_HR['MaritalStatus'].unique()):
    ratio = df_HR[(df_HR['MaritalStatus']==field)&(df_HR['Attrition']=="Yes")]
    df_Marital.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_MF = df_Marital.groupby(by="Marital Status").sum()
df_MF.iplot(kind='bar',title='Leavers by Marital Status (%)')
```



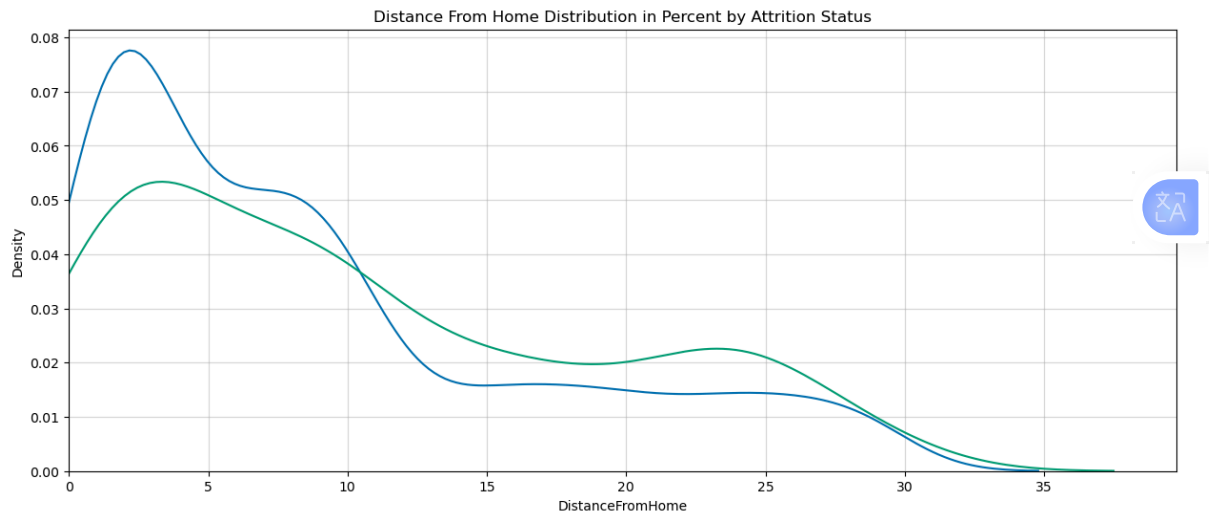
```
In [32]: # Distance from Home
print("Distance from home for employees to get to work is from {} to {} miles.
```

Distance from home for employees to get to work is from 1 to 29 miles.

```
In [33]: print('Average distance from home for currently active employees: {:.2f} miles
df_HR[df_HR['Attrition'] == 'No']['DistanceFromHome'].mean(), df_HR[df_HR[
```

Average distance from home for currently active employees: 8.92 miles and ex-employees: 10.63 miles

```
In [34]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'DistanceFromHome'], label = 'No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'DistanceFromHome'], label = 'Yes')
plt.xlabel('DistanceFromHome')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Distance From Home Distribution in Percent by Attrition Status');
```



```
In [35]: # The organisation consists of several departments
df_HR['Department'].value_counts()
```

```
Out[35]: Department
Research & Development    961
Sales                     446
Human Resources           63
Name: count, dtype: int64
```

```
In [36]: df_Department = pd.DataFrame(columns=["Department", "% of Leavers"])
i=0
for field in list(df_HR['Department'].unique()):
    ratio = df_HR[(df_HR['Department']==field)&(df_HR['Attrition']=="Yes")].size
    df_Department.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_DF = df_Department.groupby(by="Department").sum()
df_DF.iplot(kind='bar',title='Leavers by Department (%)')
```



```
In [37]: # Employees have different business travel commitment depending on their role
df_HR['BusinessTravel'].value_counts()
```

```
Out[37]: BusinessTravel
Travel_Rarely      1043
Travel_Frequently   277
Non-Travel         150
Name: count, dtype: int64
```

```
In [38]: df_BusinessTravel = pd.DataFrame(columns=["Business Travel", "% of Leavers"])
i=0
for field in list(df_HR['BusinessTravel'].unique()):
    ratio = df_HR[(df_HR['BusinessTravel']==field)&(df_HR['Attrition']=="Yes")]
    df_BusinessTravel.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_BT = df_BusinessTravel.groupby(by="Business Travel").sum()
df_BT.iplot(kind='bar',title='Leavers by Business Travel (%)')
```



```
In [39]: # Employees in the database have several roles on-file
df_HR['JobRole'].value_counts()
```

```
Out[39]: JobRole
Sales Executive          326
Research Scientist       292
Laboratory Technician    259
Manufacturing Director   145
Healthcare Representative 131
Manager                  102
Sales Representative      83
Research Director        80
Human Resources          52
Name: count, dtype: int64
```

```
In [40]: df_JobRole = pd.DataFrame(columns=["Job Role", "% of Leavers"])
i=0
for field in list(df_HR['JobRole'].unique()):
    ratio = df_HR[(df_HR['JobRole']==field)&(df_HR['Attrition']=="Yes")].shape[0]
    df_JobRole.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_JR = df_JobRole.groupby(by="Job Role").sum()
df_JR.iplot(kind='bar',title='Leavers by Job Role (%)')
```



```
In [41]: df_HR['JobLevel'].value_counts()
```

```
Out[41]: JobLevel
1      543
2      534
3      218
4      106
5        69
Name: count, dtype: int64
```

```
In [42]: df_JobLevel = pd.DataFrame(columns=["Job Level", "% of Leavers"])
i=0
for field in list(df_HR['JobLevel'].unique()):
    ratio = df_HR[(df_HR['JobLevel']==field)&(df_HR['Attrition']=="Yes")].shape[0]/df_HR.shape[0]
    df_JobLevel.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_JL = df_JobLevel.groupby(by="Job Level").sum()
df_JL.iplot(kind='bar',title='Leavers by Job Level (%)')
```



```
In [43]: df_HR['JobInvolvement'].value_counts()
```

```
Out[43]: JobInvolvement
3      868
2      375
4      144
1       83
Name: count, dtype: int64
```

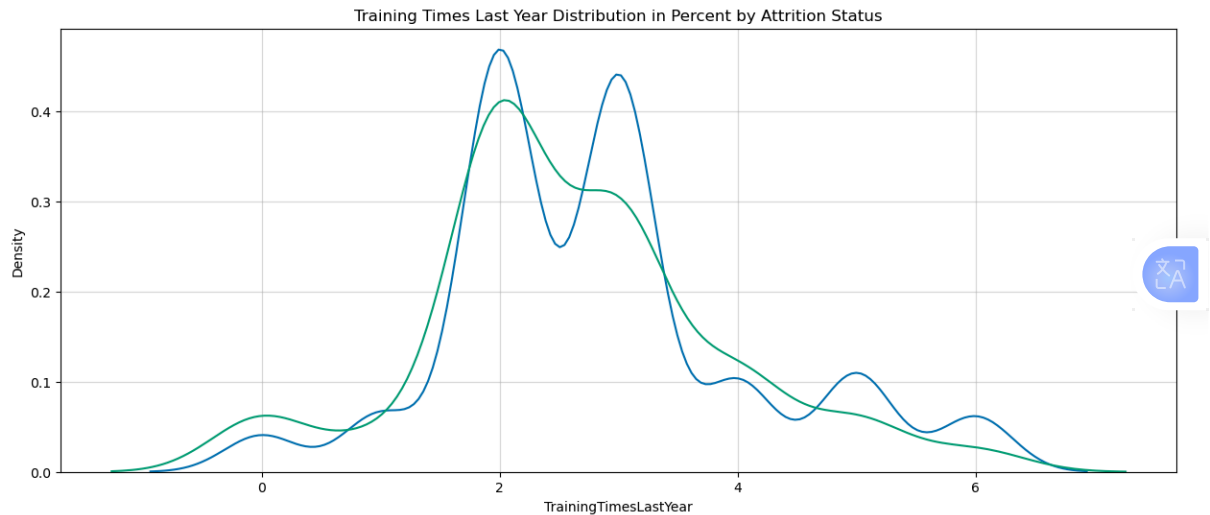
```
In [44]: df_JobInvolvement = pd.DataFrame(columns=["Job Involvement", "% of Leavers"])
i=0
for field in list(df_HR['JobInvolvement'].unique()):
    ratio = df_HR[(df_HR['JobInvolvement']==field)&(df_HR['Attrition']=="Yes")]
    df_JobInvolvement.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_JI = df_JobInvolvement.groupby(by="Job Involvement").sum()
df_JI.iplot(kind='bar',title='Leavers by Job Involvement (%)')
```



```
In [45]: print("Number of training times last year varies from {} to {} years.".format(
    df_HR['TrainingTimesLastYear'].min(), df_HR['TrainingTimesLastYear'].max())
```

Number of training times last year varies from 0 to 6 years.

```
In [46]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'TrainingTimesLastYear'], label='No Attrition')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'TrainingTimesLastYear'], label='Yes Attrition')
plt.xlabel('TrainingTimesLastYear')
plt.ylabel('Density')
plt.title('Training Times Last Year Distribution in Percent by Attrition Status')
```



```
In [47]: df_HR['NumCompaniesWorked'].value_counts()
```

```
Out[47]: NumCompaniesWorked
1      521
0      197
3      159
2      146
4      139
7       74
6       70
5       63
9       52
8       49
Name: count, dtype: int64
```



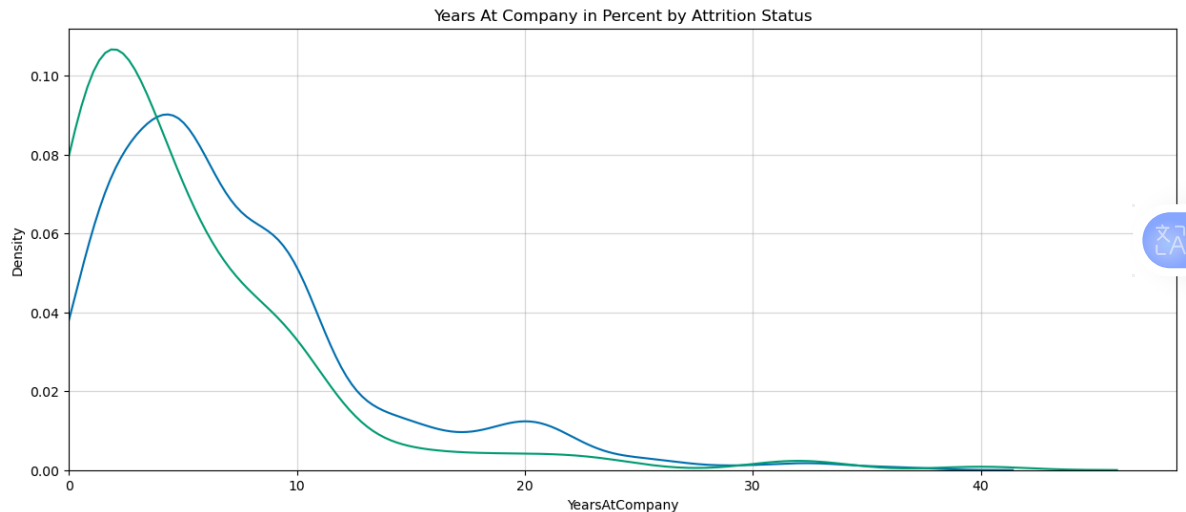
```
In [48]: df_NumCompaniesWorked = pd.DataFrame(columns=["Num Companies Worked", "% of Leavers"])
i=0
for field in list(df_HR['NumCompaniesWorked'].unique()):
    ratio = df_HR[(df_HR['NumCompaniesWorked']==field)&(df_HR['Attrition']=="Yes")].count()/df_HR.count()
    df_NumCompaniesWorked.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_NC = df_NumCompaniesWorked.groupby(by="Num Companies Worked").sum()
df_NC.iplot(kind='bar',title='Leavers by Num Companies Worked (%)')
```



```
In [49]: print("Number of Years at the company varies from {} to {} years.".format(
    df_HR['YearsAtCompany'].min(), df_HR['YearsAtCompany'].max()))
```

Number of Years at the company varies from 0 to 40 years.

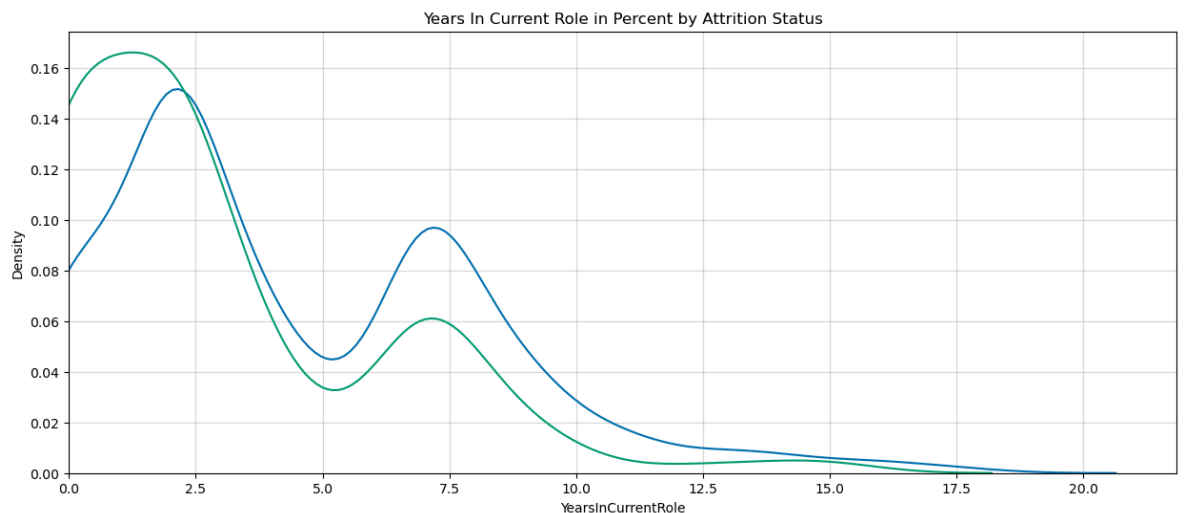
```
In [50]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'YearsAtCompany'], label = 'No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'YearsAtCompany'], label = 'Yes')
plt.xlabel('YearsAtCompany')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Years At Company in Percent by Attrition Status');
```



```
In [51]: print("Number of Years in the current role varies from {} to {} years.".format(
df_HR['YearsInCurrentRole'].min(), df_HR['YearsInCurrentRole'].max()))
```

Number of Years in the current role varies from 0 to 18 years.

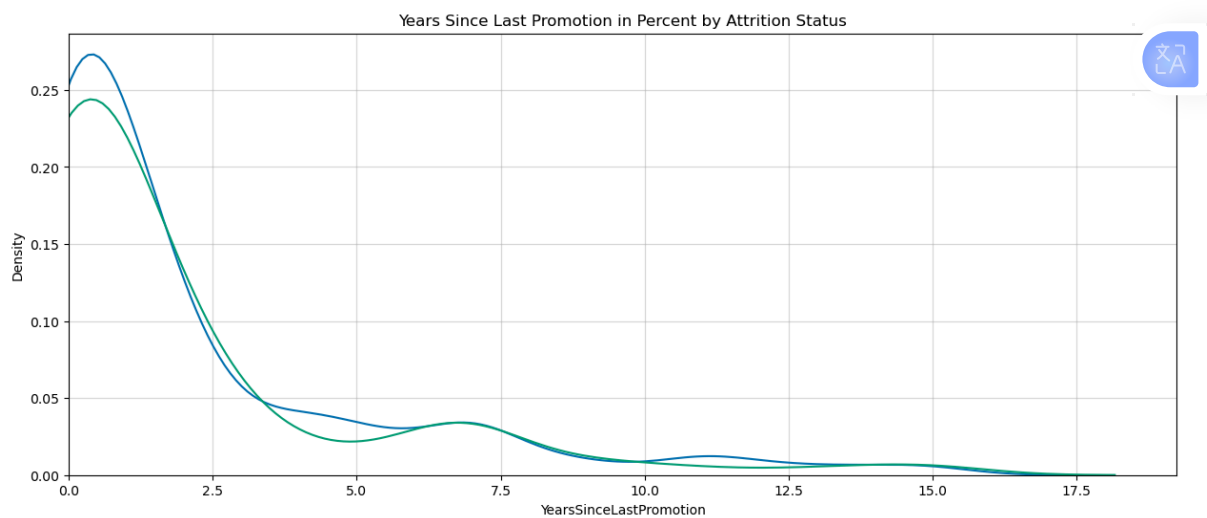
```
In [52]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'YearsInCurrentRole'], label = 'No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'YearsInCurrentRole'], label = 'Yes')
plt.xlabel('YearsInCurrentRole')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Years In Current Role in Percent by Attrition Status');
```



```
In [53]: print("Number of Years since last promotion varies from {} to {} years.".format(
          df_HR['YearsSinceLastPromotion'].min(), df_HR['YearsSinceLastPromotion'].max()))
```

Number of Years since last promotion varies from 0 to 15 years.

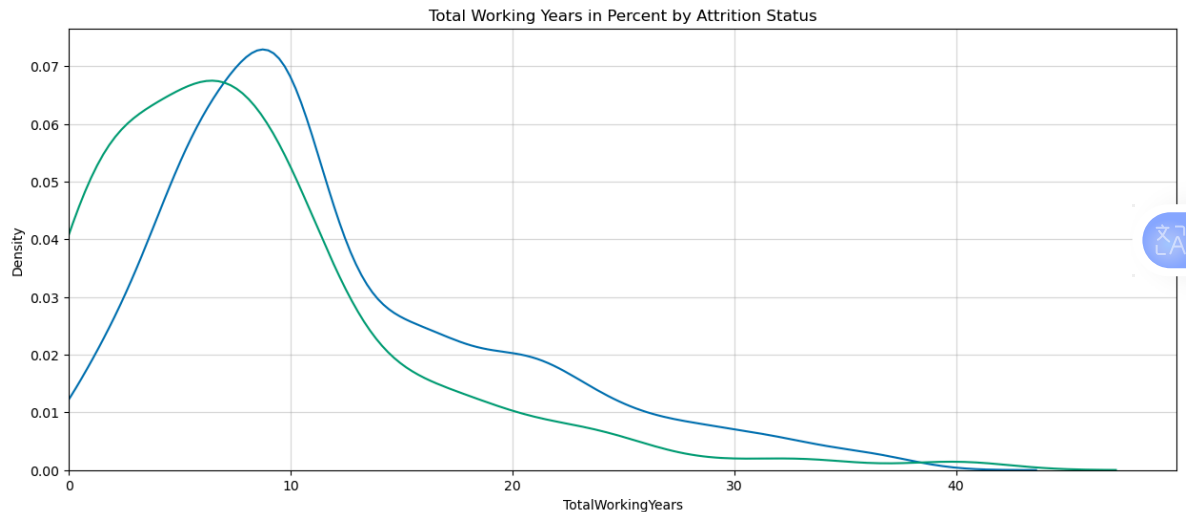
```
In [54]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'YearsSinceLastPromotion'],
            df_HR.loc[df_HR['Attrition'] == 'Yes', 'YearsSinceLastPromotion'],
            plt.xlabel('YearsSinceLastPromotion')
            plt.xlim(left=0)
            plt.ylabel('Density')
            plt.title('Years Since Last Promotion in Percent by Attrition Status');
```



```
In [55]: print("Total working years varies from {} to {} years.".format(
          df_HR['TotalWorkingYears'].min(), df_HR['TotalWorkingYears'].max()))
```

Total working years varies from 0 to 40 years.

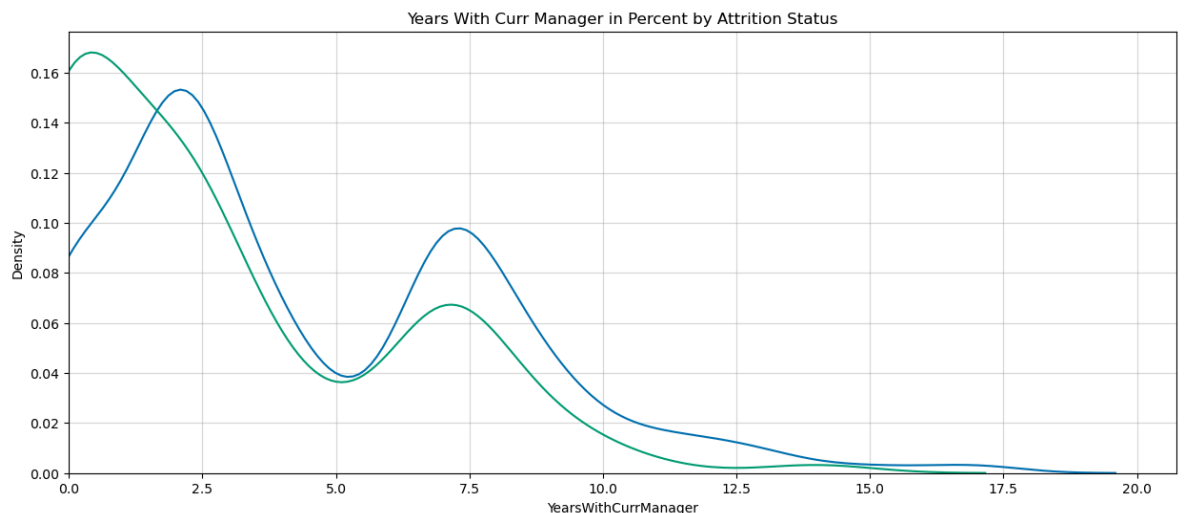
```
In [56]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'TotalWorkingYears'], label='No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'TotalWorkingYears'], label='Yes')
plt.xlabel('TotalWorkingYears')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Total Working Years in Percent by Attrition Status');
```



```
In [57]: print("Number of Years wit current manager varies from {} to {} years.".format(
df_HR['YearsWithCurrManager'].min(), df_HR['YearsWithCurrManager'].max()))
```

Number of Years wit current manager varies from 0 to 17 years.

```
In [58]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'YearsWithCurrManager'], label='No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'YearsWithCurrManager'], label='Yes')
plt.xlabel('YearsWithCurrManager')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Years With Curr Manager in Percent by Attrition Status');
```



```
In [59]: df_HR['WorkLifeBalance'].value_counts()
```

```
Out[59]: WorkLifeBalance
3      893
2      344
4      153
1       80
Name: count, dtype: int64
```

```
In [60]: df_WorkLifeBalance = pd.DataFrame(columns=["WorkLifeBalance", "% of Leavers"])
i=0
for field in list(df_HR['WorkLifeBalance'].unique()):
    ratio = df_HR[(df_HR['WorkLifeBalance']==field)&(df_HR['Attrition']=="Yes")]
    df_WorkLifeBalance.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_WLB = df_WorkLifeBalance.groupby(by="WorkLifeBalance").sum()
df_WLB.iplot(kind='bar',title='Leavers by WorkLifeBalance (%)')
```

```
In [61]: df_HR['StandardHours'].value_counts()
```

```
Out[61]: StandardHours
80      1470
Name: count, dtype: int64
```

```
In [62]: df_HR['OverTime'].value_counts()
```

```
Out[62]: OverTime
No      1054
Yes      416
Name: count, dtype: int64
```

```
In [63]: df_OverTime = pd.DataFrame(columns=["OverTime", "% of Leavers"])
i=0
for field in list(df_HR['OverTime'].unique()):
    ratio = df_HR[(df_HR['OverTime']==field)&(df_HR['Attrition']=="Yes")].shape[0]
    df_OverTime.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_OT = df_OverTime.groupby(by="OverTime").sum()
df_OT.iplot(kind='bar',title='Leavers by OverTime (%)')
```



```
In [64]: print("Employee Hourly Rate varies from ${} to ${}.".format(
    df_HR['HourlyRate'].min(), df_HR['HourlyRate'].max()))
```

Employee Hourly Rate varies from \$30 to \$100.

```
In [65]: print("Employee Daily Rate varies from ${} to ${}.".format(
    df_HR['DailyRate'].min(), df_HR['DailyRate'].max()))
```

Employee Daily Rate varies from \$102 to \$1499.

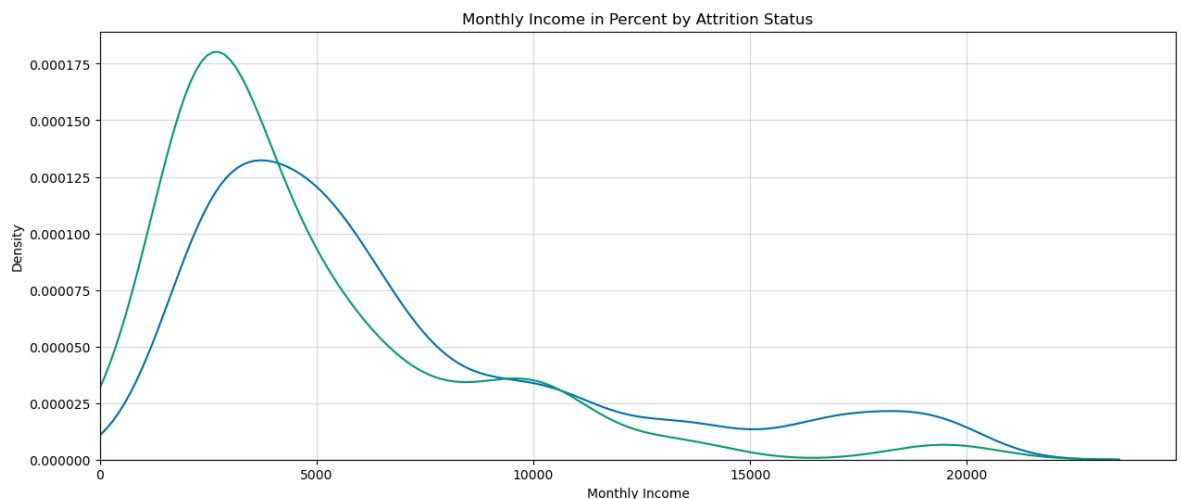
```
In [66]: print("Employee Monthly Rate varies from ${} to ${}.".format(  
        df_HR['MonthlyRate'].min(), df_HR['MonthlyRate'].max()))
```

Employee Monthly Rate varies from \$2094 to \$26999.

```
In [67]: print("Employee Monthly Income varies from ${} to ${}.".format(  
        df_HR['MonthlyIncome'].min(), df_HR['MonthlyIncome'].max()))
```

Employee Monthly Income varies from \$1009 to \$19999.

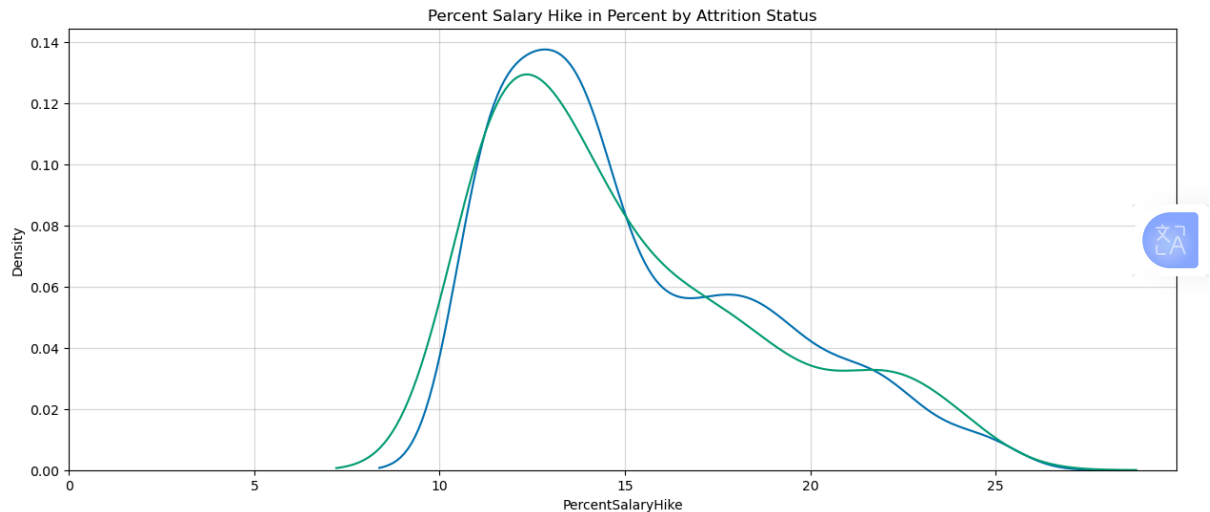
```
In [68]: plt.figure(figsize=(15,6))  
plt.style.use('seaborn-colorblind')  
plt.grid(True, alpha=0.5)  
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'MonthlyIncome'], label = 'A')  
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'MonthlyIncome'], label = 'B')  
plt.xlabel('Monthly Income')  
plt.xlim(left=0)  
plt.ylabel('Density')  
plt.title('Monthly Income in Percent by Attrition Status');
```



```
In [69]: print("Percentage Salary Hikes varies from {}% to {}%.".format(  
        df_HR['PercentSalaryHike'].min(), df_HR['PercentSalaryHike'].max()))
```

Percentage Salary Hikes varies from 11% to 25%.

```
In [70]: plt.figure(figsize=(15,6))
plt.style.use('seaborn-colorblind')
plt.grid(True, alpha=0.5)
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'No', 'PercentSalaryHike'], label='No')
sns.kdeplot(df_HR.loc[df_HR['Attrition'] == 'Yes', 'PercentSalaryHike'], label='Yes')
plt.xlabel('PercentSalaryHike')
plt.xlim(left=0)
plt.ylabel('Density')
plt.title('Percent Salary Hike in Percent by Attrition Status');
```



```
In [71]: print("Stock Option Levels varies from {} to {}".format(
    df_HR['StockOptionLevel'].min(), df_HR['StockOptionLevel'].max()))
```

Stock Option Levels varies from 0 to 3.

```
In [72]: print("Normalised percentage of leavers by Stock Option Level: 1: {:.2f}%, 2: {:.2f}%, 3: {:.2f}%".format(
    df_HR[(df_HR['Attrition'] == 'Yes') & (df_HR['StockOptionLevel'] == 1)].shape[0] / df_HR[df_HR['Attrition'] == 'Yes'].shape[0]*100,
    df_HR[(df_HR['Attrition'] == 'Yes') & (df_HR['StockOptionLevel'] == 2)].shape[0] / df_HR[df_HR['Attrition'] == 'Yes'].shape[0]*100,
    df_HR[(df_HR['Attrition'] == 'Yes') & (df_HR['StockOptionLevel'] == 3)].shape[0] / df_HR[df_HR['Attrition'] == 'Yes'].shape[0]*100))
```

Normalised percentage of leavers by Stock Option Level: 1: 9.40%, 2: 2.01%, 3: 2.52%


```
In [73]: df_StockOptionLevel = pd.DataFrame(columns=["StockOptionLevel", "% of Leavers"])
i=0
for field in list(df_HR['StockOptionLevel'].unique()):
    ratio = df_HR[(df_HR['StockOptionLevel']==field)&(df_HR['Attrition']=="Yes")].count().sum()
    df_StockOptionLevel.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_SOL = df_StockOptionLevel.groupby(by="StockOptionLevel").sum()
df_SOL.iplot(kind='bar',title='Leavers by Stock Option Level (%)')
```



```
In [74]: df_HR['EnvironmentSatisfaction'].value_counts()
```

```
Out[74]: EnvironmentSatisfaction
3      453
4      446
2      287
1      284
Name: count, dtype: int64
```

```
In [75]: df_EnvironmentSatisfaction = pd.DataFrame(columns=["EnvironmentSatisfaction",
i=0
for field in list(df_HR['EnvironmentSatisfaction'].unique()):
    ratio = df_HR[(df_HR['EnvironmentSatisfaction']==field)&(df_HR['Attrition'
df_EnvironmentSatisfaction.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_Env = df_EnvironmentSatisfaction.groupby(by="EnvironmentSatisfaction").sum(
df_Env.iplot(kind='bar',title='Leavers by Environment Satisfaction (%)')
```



```
In [76]: # Job Satisfaction was captured as: 1 'Low' 2 'Medium' 3 'High' 4 'Very High'
df_HR['JobSatisfaction'].value_counts()
```

```
Out[76]: JobSatisfaction
4      459
3      442
1      289
2      280
Name: count, dtype: int64
```

```
In [77]: df_JobSatisfaction = pd.DataFrame(columns=["JobSatisfaction", "% of Leavers"])
i=0
for field in list(df_HR['JobSatisfaction'].unique()):
    ratio = df_HR[(df_HR['JobSatisfaction']==field)&(df_HR['Attrition']=="Yes")]
    df_JobSatisfaction.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_JS = df_JobSatisfaction.groupby(by="JobSatisfaction").sum()
df_JS.iplot(kind='bar',title='Leavers by Job Satisfaction (%)')
```



```
In [78]: df_HR['RelationshipSatisfaction'].value_counts()
```

```
Out[78]: RelationshipSatisfaction
3      459
4      432
2      303
1      276
Name: count, dtype: int64
```

```
In [79]: df_RelationshipSatisfaction = pd.DataFrame(columns=["RelationshipSatisfaction"]
i=0
for field in list(df_HR['RelationshipSatisfaction'].unique()):
    ratio = df_HR[(df_HR['RelationshipSatisfaction']==field)&(df_HR['Attrition']=='Yes')]
    df_RelationshipSatisfaction.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_RS = df_RelationshipSatisfaction.groupby(by="RelationshipSatisfaction").sum()
df_RS.iplot(kind='bar',title='Leavers by Relationship Satisfaction (%)')
```



```
In [80]: df_HR['PerformanceRating'].value_counts()
```

```
Out[80]: PerformanceRating
3      1244
4       226
Name: count, dtype: int64
```

```
In [81]: print("Normalised percentage of leavers by Stock Option Level: 3: {:.2f}%, 4: {:.2f}%".format(
df_HR[(df_HR['Attrition']=='Yes') & (df_HR['PerformanceRating']==3)].shape[0] / df_HR[df_HR['StockOptionLevel']==1].shape[0]*100,
df_HR[(df_HR['Attrition']=='Yes') & (df_HR['PerformanceRating']==4)].shape[0] / df_HR[df_HR['StockOptionLevel']==1].shape[0]*100))
```

Normalised percentage of leavers by Stock Option Level: 3: 33.56%, 4: 6.21%

```
In [82]: df_PerformanceRating = pd.DataFrame(columns=["PerformanceRating", "% of Leavers"])
i=0
for field in list(df_HR['PerformanceRating'].unique()):
    ratio = df_HR[(df_HR['PerformanceRating']==field)&(df_HR['Attrition']=="Yes")].shape[0] / df_HR.shape[0]
    df_PerformanceRating.loc[i] = (field, ratio*100)
    i += 1
    #print("In {}, the ratio of leavers is {:.2f}%".format(field, ratio*100))
df_PR = df_PerformanceRating.groupby(by="PerformanceRating").sum()
df_PR.iplot(kind='bar',title='Leavers by Performance Rating (%)')
```



```
In [83]: # Attrition indicates if the employee is currently active ('No') or has left the company ('Yes')
df_HR['Attrition'].value_counts()
```

```
Out[83]: Attrition
No      1233
Yes      237
Name: count, dtype: int64
```

```
In [84]: print("Percentage of Current Employees is {:.1f}% and of Ex-employees is: {:.1f}%".format(
    df_HR[df_HR['Attrition'] == 'No'].shape[0] / df_HR.shape[0]*100,
    df_HR[df_HR['Attrition'] == 'Yes'].shape[0] / df_HR.shape[0]*100))
```

Percentage of Current Employees is 83.9% and of Ex-employees is: 16.1%

```
In [85]: df_HR['Attrition'].iplot(kind='hist', xTitle='Attrition',  
                                   yTitle='count', title='Attrition Distribution')
```



```
In [86]: # Find correlations with the target and sort
df_HR_trans = df_HR.copy()

df_HR_trans['Target'] = df_HR_trans['Attrition'].apply(
    lambda x: 0 if x == 'No' else 1
)

df_HR_trans = df_HR_trans.drop(
    ['Attrition', 'EmployeeCount', 'EmployeeNumber', 'StandardHours', 'Over18',
     axis=1
)

# 🐞 FIX: use numeric columns only
correlations = (
    df_HR_trans
    .select_dtypes(include='number')
    .corr()['Target']
    .sort_values()
)

print('Most Positive Correlations:\n', correlations.tail(5))
print('\nMost Negative Correlations:\n', correlations.head(5))
```

Most Positive Correlations:

PerformanceRating	0.002889
MonthlyRate	0.015170
NumCompaniesWorked	0.043494
DistanceFromHome	0.077924
Target	1.000000

Name: Target, dtype: float64

Most Negative Correlations:

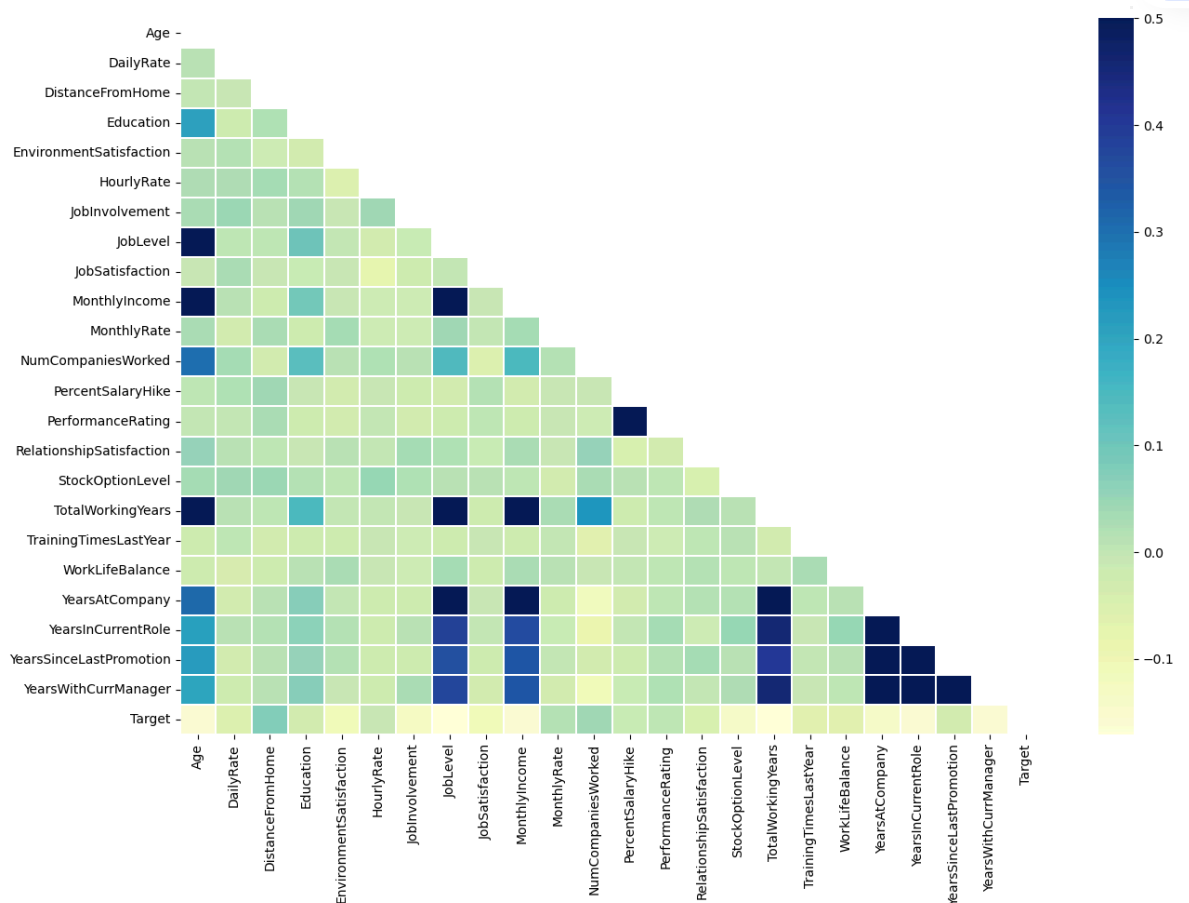
TotalWorkingYears	-0.171063
JobLevel	-0.169105
YearsInCurrentRole	-0.160545
MonthlyIncome	-0.159840
Age	-0.159205

Name: Target, dtype: float64

```
In [87]: # Calculate correlations (numeric columns only)
corr = df_HR_trans.select_dtypes(include='number').corr()

mask = np.zeros_like(corr)
mask[np.triu_indices_from(mask)] = True

# Heatmap
plt.figure(figsize=(15, 10))
sns.heatmap(
    corr,
    vmax=0.5,
    mask=mask,
    linewidths=0.2,
    cmap="YlGnBu"
)
plt.show()
```



```
In [88]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder
# Create a Label encoder object
le = LabelEncoder()
```



```
In [89]: print(df_HR.shape)
df_HR.head()
```

(1470, 35)

```
Out[89]:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educ
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Lif
1	49	No	Travel_Frequently	279	Research & Development	8	1	Lif
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Lif
4	27	No	Travel_Rarely	591	Research & Development	2	1	

```
In [90]: # Label Encoding will be used for columns with 2 or less unique values
le_count = 0
for col in df_HR.columns[1:]:
    if df_HR[col].dtype == 'object':
        if len(list(df_HR[col].unique())) <= 2:
            le.fit(df_HR[col])
            df_HR[col] = le.transform(df_HR[col])
            le_count += 1
print('{} columns were label encoded.'.format(le_count))
```

4 columns were label encoded.

```
In [91]: # convert rest of categorical variable into dummy
df_HR = pd.get_dummies(df_HR, drop_first=True)
```

```
In [92]: print(df_HR.shape)
df_HR.head()
```

(1470, 49)

```
Out[92]:
```

	Age	Attrition	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber
0	41	1	1102	1	2	1	1
1	49	0	279	8	1	1	2
2	37	1	1373	2	2	1	4
3	33	0	1392	3	4	1	5
4	27	0	591	2	1	1	7

```
In [93]: # import MinMaxScaler
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 5))
HR_col = list(df_HR.columns)
HR_col.remove('Attrition')
for col in HR_col:
    df_HR[col] = df_HR[col].astype(float)
    df_HR[[col]] = scaler.fit_transform(df_HR[[col]])
df_HR['Attrition'] = pd.to_numeric(df_HR['Attrition'], downcast='float')
df_HR.head()
```

Out[93]:

	Age	Attrition	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumt
0	2.738095	1.0	3.579098	0.000000	1.25	0.0	0.0000
1	3.690476	0.0	0.633500	1.250000	0.00	0.0	0.0024
2	2.261905	1.0	4.549034	0.178571	1.25	0.0	(
3	1.785714	0.0	4.617037	0.357143	3.75	0.0	0.0096
4	1.071429	0.0	1.750179	0.178571	0.00	0.0	0.0145

```
In [94]: print('Size of Full Encoded Dataset: {}'.format(df_HR.shape))
```

Size of Full Encoded Dataset: (1470, 49)

```
In [95]: # assign the target to a new dataframe and convert it to a numerical feature
#df_target = df_HR[['Attrition']].copy()
target = df_HR['Attrition'].copy()
```

```
In [96]: type(target)
```

Out[96]: pandas.core.series.Series

```
In [97]: # Let's remove the target feature and redundant features from the dataset
df_HR.drop(['Attrition', 'EmployeeCount', 'EmployeeNumber',
            'StandardHours', 'Over18'], axis=1, inplace=True)
print('Size of Full dataset is: {}'.format(df_HR.shape))
```

Size of Full dataset is: (1470, 44)

```
In [98]: # Since we have class imbalance (i.e. more employees with turnover=0 than turn
# Let's use stratify=y to maintain the same ratio as in the training dataset w
X_train, X_test, y_train, y_test = train_test_split(df_HR,
                                                    target,
                                                    test_size=0.25,
                                                    random_state=7,
                                                    stratify=target)

print("Number transactions X_train dataset: ", X_train.shape)
print("Number transactions y_train dataset: ", y_train.shape)
print("Number transactions X_test dataset: ", X_test.shape)
print("Number transactions y_test dataset: ", y_test.shape)
```

```
Number transactions X_train dataset: (1102, 44)
Number transactions y_train dataset: (1102,)
Number transactions X_test dataset: (368, 44)
Number transactions y_test dataset: (368,)
```

```
In [99]: # selection of algorithms to consider and set performance measure
models = []
models.append(('Logistic Regression', LogisticRegression(solver='liblinear', r
                                                         class_weight='balance

models.append(('Random Forest', RandomForestClassifier(
    n_estimators=100, random_state=7)))
models.append(('SVM', SVC(gamma='auto', random_state=7)))
models.append(('KNN', KNeighborsClassifier()))
models.append(('Decision Tree Classifier',
               DecisionTreeClassifier(random_state=7)))
models.append(('Gaussian NB', GaussianNB()))
```

```

In [100]: acc_results = []
auc_results = []
names = []

# set table to populate performance results
col = ['Algorithm', 'ROC AUC Mean', 'ROC AUC STD',
        'Accuracy Mean', 'Accuracy STD']
df_results = pd.DataFrame(columns=col)

i = 0

# evaluate each model using cross-validation
for name, model in models:
    kfold = model_selection.KFold(
        n_splits=10, shuffle=True, random_state=7
    )

    cv_acc_results = model_selection.cross_val_score(
        model, X_train, y_train, cv=kfold, scoring='accuracy'
    )

    cv_auc_results = model_selection.cross_val_score(
        model, X_train, y_train, cv=kfold, scoring='roc_auc'
    )

    acc_results.append(cv_acc_results)
    auc_results.append(cv_auc_results)
    names.append(name)

    df_results.loc[i] = [
        name,
        round(cv_auc_results.mean() * 100, 2),
        round(cv_auc_results.std() * 100, 2),
        round(cv_acc_results.mean() * 100, 2),
        round(cv_acc_results.std() * 100, 2)
    ]
    i += 1

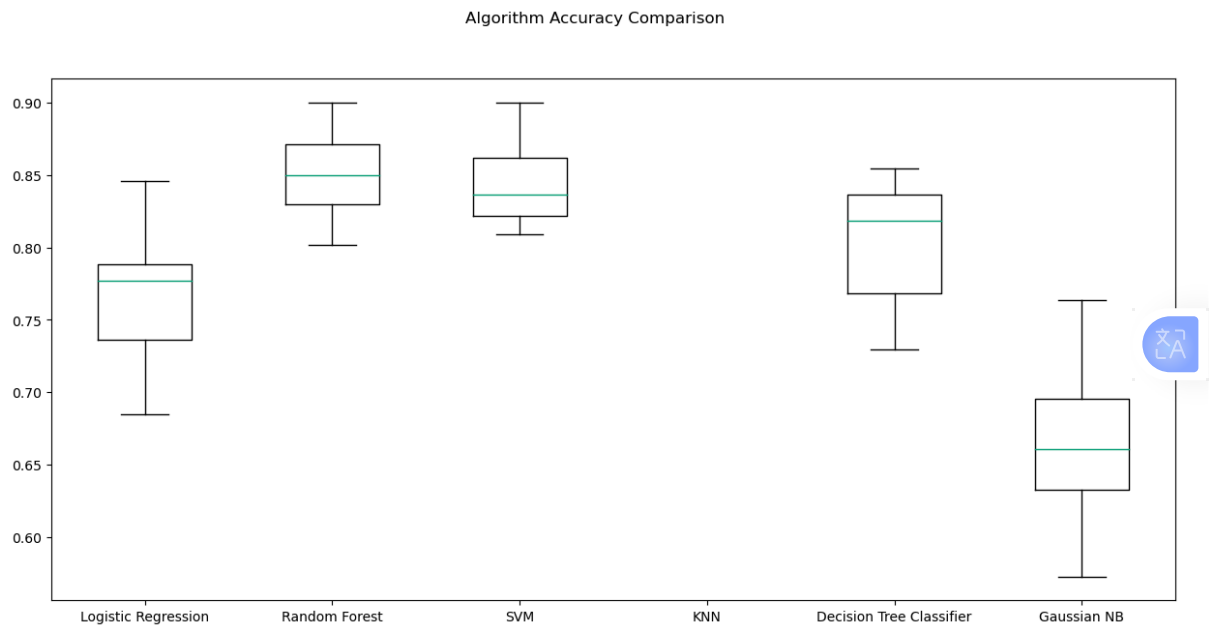
df_results.sort_values(by='ROC AUC Mean', ascending=False)

```

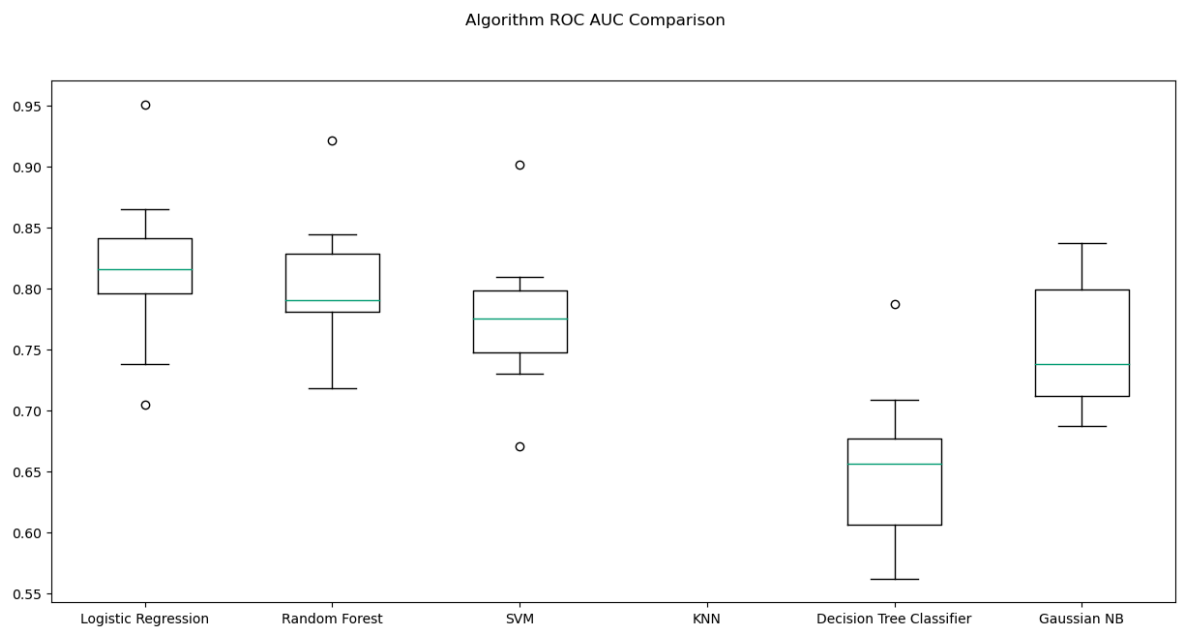
Out[100]:

	Algorithm	ROC AUC Mean	ROC AUC STD	Accuracy Mean	Accuracy STD
0	Logistic Regression	81.64	6.43	76.51	4.36
1	Random Forest	80.03	5.55	85.12	2.86
2	SVM	77.63	5.71	84.30	2.66
5	Gaussian NB	75.11	5.02	66.33	4.99
4	Decision Tree Classifier	65.37	6.30	80.31	4.05
3	KNN	NaN	NaN	NaN	NaN

```
In [101]: fig = plt.figure(figsize=(15, 7))
fig.suptitle('Algorithm Accuracy Comparison')
ax = fig.add_subplot(111)
plt.boxplot(acc_results)
ax.set_xticklabels(names)
plt.show()
```



```
In [102]: fig = plt.figure(figsize=(15, 7))
fig.suptitle('Algorithm ROC AUC Comparison')
ax = fig.add_subplot(111)
plt.boxplot(auc_results)
ax.set_xticklabels(names)
plt.show()
```



```
In [103]: kfold = model_selection.KFold(
            n_splits=10,
            shuffle=True,      #  REQUIRED
            random_state=7
        )

        modelCV = LogisticRegression(
            solver='liblinear',
            class_weight="balanced",
            random_state=7
        )

        scoring = 'roc_auc'

        results = model_selection.cross_val_score(
            modelCV, X_train, y_train, cv=kfold, scoring=scoring
        )

        print("AUC score (STD): %.2f (%.2f)" % (results.mean(), results.std()))
```

AUC score (STD): 0.82 (0.06)

```
In [104]: param_grid = {'C': np.arange(1e-03, 2, 0.01)} # hyper-parameter list to fine-

        log_gs = GridSearchCV(
            LogisticRegression(
                solver='liblinear',
                class_weight="balanced",
                random_state=7
            ),
            return_train_score=True,
            param_grid=param_grid,
            scoring='roc_auc',
            cv=10
        )

        log_grid = log_gs.fit(X_train, y_train)
        log_opt = log_grid.best_estimator_
        results = log_gs.cv_results_

        print('=' * 20)
        print("best estimator:", log_gs.best_estimator_)
        print("best params:", log_gs.best_params_)
        print('best score:', log_gs.best_score_)
        print('=' * 20)
```

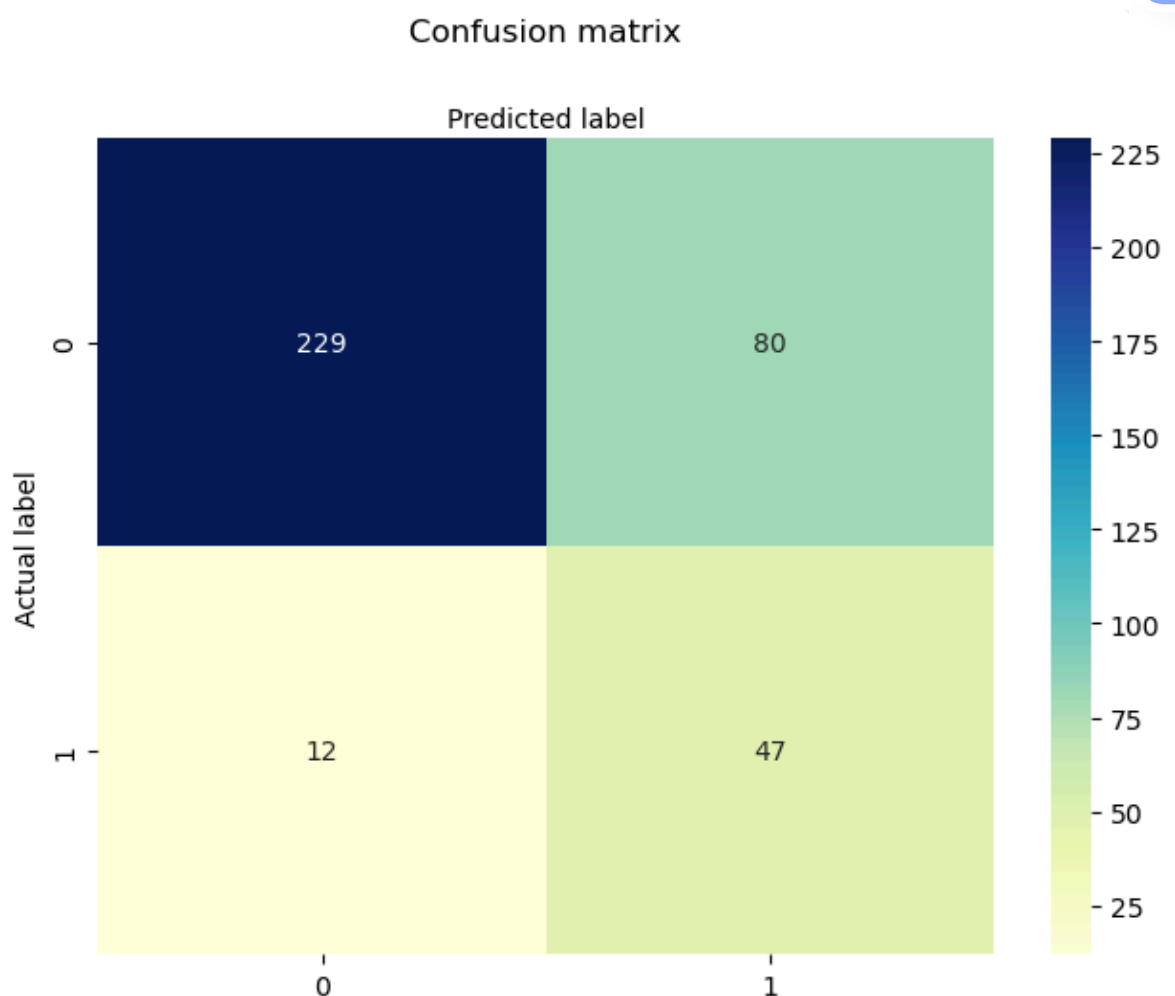
```
=====
best estimator: LogisticRegression(C=0.10099999999999998, class_weight='balanced',
                                   random_state=7, solver='liblinear')
best params: {'C': 0.10099999999999998}
best score: 0.8154962538233308
=====
```

```

In [105]: ## Confusion Matrix
cnf_matrix = metrics.confusion_matrix(y_test, log_opt.predict(X_test))
class_names=[0,1] # name of classes
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# create heatmap
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True, cmap="YlGnBu", fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Confusion matrix', y=1.1)
plt.ylabel('Actual label')
plt.xlabel('Predicted label')

```

Out[105]: Text(0.5, 427.9555555555555, 'Predicted label')



```

In [106]: print('Accuracy of Logistic Regression Classifier on test set: {:.2f}'.format(

```

Accuracy of Logistic Regression Classifier on test set: 75.00

```
In [107]: # Classification report for the optimised Log Regression
log_opt.fit(X_train, y_train)
print(classification_report(y_test, log_opt.predict(X_test)))
```

	precision	recall	f1-score	support
0.0	0.95	0.74	0.83	309
1.0	0.37	0.80	0.51	59
accuracy			0.75	368
macro avg	0.66	0.77	0.67	368
weighted avg	0.86	0.75	0.78	368

```
In [108]: log_opt.fit(X_train, y_train) # fit optimised model to the training data
probs = log_opt.predict_proba(X_test) # predict probabilities
probs = probs[:, 1] # we will only keep probabilities associated with the
logit_roc_auc = roc_auc_score(y_test, probs) # calculate AUC score using test
print('AUC score: %.3f' % logit_roc_auc)
```

AUC score: 0.857


```
In [109]: rf_classifier = RandomForestClassifier(
            class_weight="balanced",
            random_state=7
        )

        param_grid = {
            'n_estimators': [50, 75, 100, 125, 150, 175],
            'min_samples_split': [2, 4, 6, 8, 10],
            'min_samples_leaf': [1, 2, 3, 4],
            'max_depth': [5, 10, 15, 20, 25]
        }

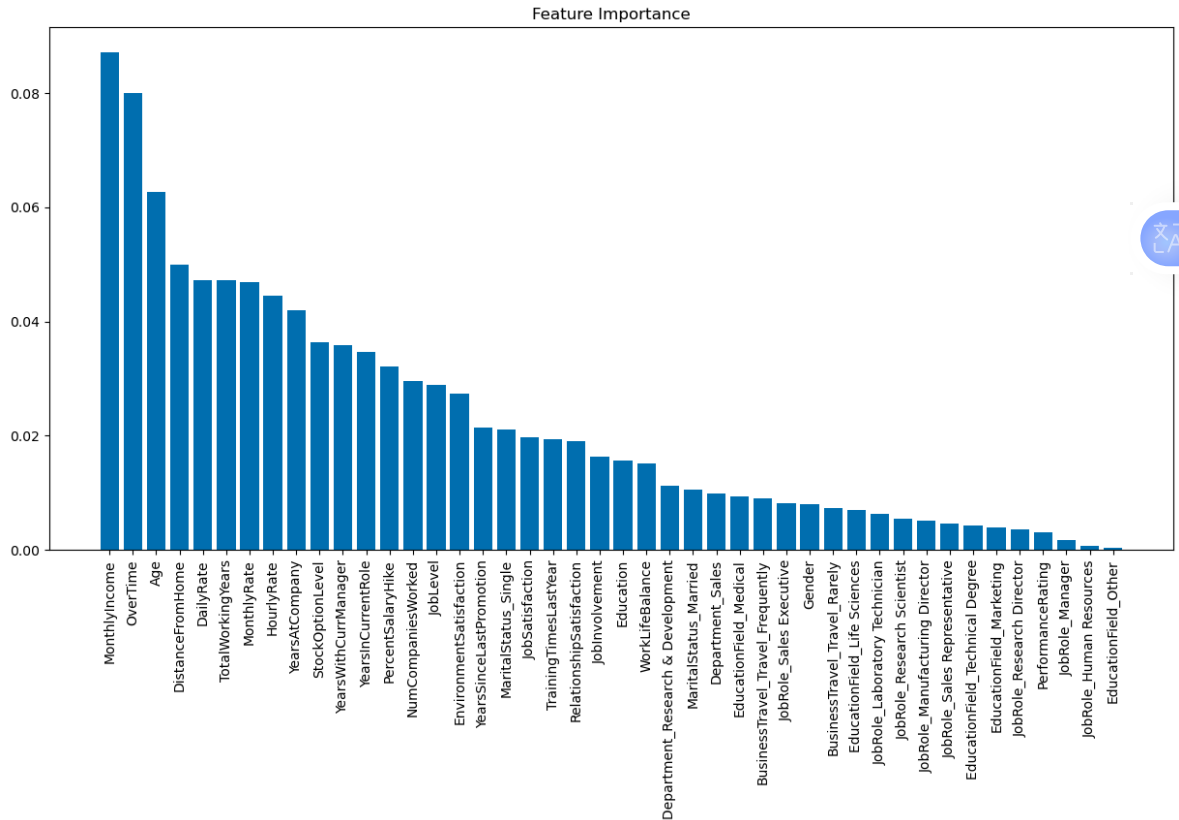
        grid_obj = GridSearchCV(
            rf_classifier,
            return_train_score=True,
            param_grid=param_grid,
            scoring='roc_auc',
            cv=10
        )

        grid_fit = grid_obj.fit(X_train, y_train)
        rf_opt = grid_fit.best_estimator_

        print('=' * 20)
        print("best estimator:", grid_obj.best_estimator_)
        print("best params:", grid_obj.best_params_)
        print('best score:', grid_obj.best_score_)
        print('=' * 20)
```

```
=====
best estimator: RandomForestClassifier(class_weight='balanced', max_depth=1
5,
                                min_samples_leaf=3, min_samples_split=8, random_state
=7)
best params: {'max_depth': 15, 'min_samples_leaf': 3, 'min_samples_split':
8, 'n_estimators': 100}
best score: 0.792510434904833
=====
```

```
In [110]: importances = rf_opt.feature_importances_
indices = np.argsort(importances)[::-1] # Sort feature importances in descending order
names = [X_train.columns[i] for i in indices] # Rearrange feature names so they match the order of importance
plt.figure(figsize=(15, 7)) # Create plot
plt.title("Feature Importance") # Create plot title
plt.bar(range(X_train.shape[1]), importances[indices]) # Add bars
plt.xticks(range(X_train.shape[1]), names, rotation=90) # Add feature names as ticks
plt.show() # Show plot
```



```
In [111]: importances = rf_opt.feature_importances_  
df_param_coeff = pd.DataFrame(columns=['Feature', 'Coefficient'])  
for i in range(44):  
    feat = X_train.columns[i]  
    coeff = importances[i]  
    df_param_coeff.loc[i] = (feat, coeff)  
df_param_coeff.sort_values(by='Coefficient', ascending=False, inplace=True)  
df_param_coeff = df_param_coeff.reset_index(drop=True)  
df_param_coeff.head(10)
```

Out[111]:

	Feature	Coefficient
0	MonthlyIncome	0.087139
1	OverTime	0.080039
2	Age	0.062667
3	DistanceFromHome	0.049933
4	DailyRate	0.047296
5	TotalWorkingYears	0.047254
6	MonthlyRate	0.046920
7	HourlyRate	0.044520
8	YearsAtCompany	0.041950
9	StockOptionLevel	0.036293

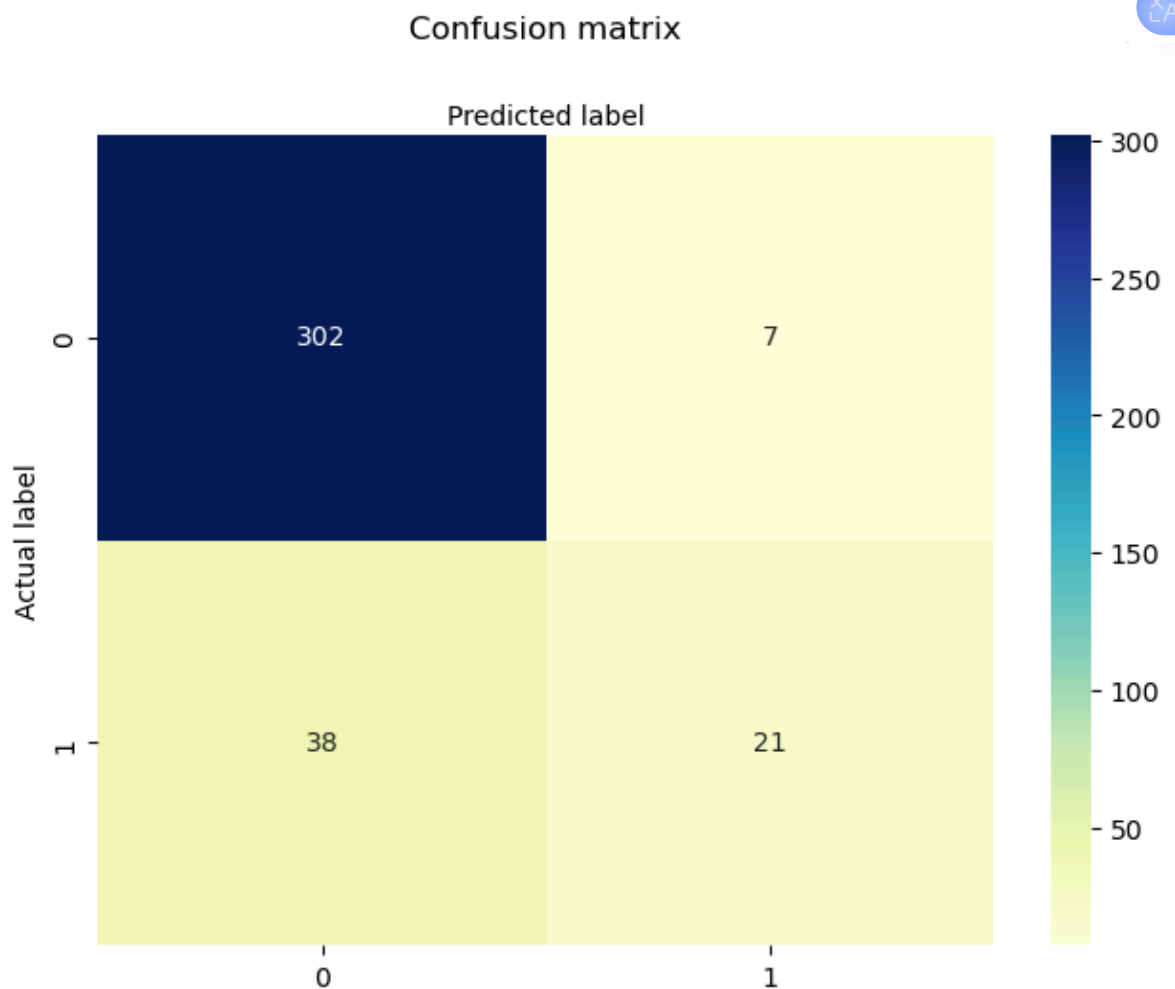


```

In [112]: ## Confusion Matrix
cnf_matrix = metrics.confusion_matrix(y_test, rf_opt.predict(X_test))
class_names=[0,1] # name of classes
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# create heatmap
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True, cmap="YlGnBu", fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Confusion matrix', y=1.1)
plt.ylabel('Actual label')
plt.xlabel('Predicted label')

```

Out[112]: Text(0.5, 427.9555555555555, 'Predicted label')



```

In [113]: print('Accuracy of RandomForest Regression Classifier on test set: {:.2f}'.for
Accuracy of RandomForest Regression Classifier on test set: 87.77

```

```
In [114]: # Classification report for the optimised RF Regression
rf_opt.fit(X_train, y_train)
print(classification_report(y_test, rf_opt.predict(X_test)))
```

	precision	recall	f1-score	support
0.0	0.89	0.98	0.93	309
1.0	0.75	0.36	0.48	59
accuracy			0.88	368
macro avg	0.82	0.67	0.71	368
weighted avg	0.87	0.88	0.86	368

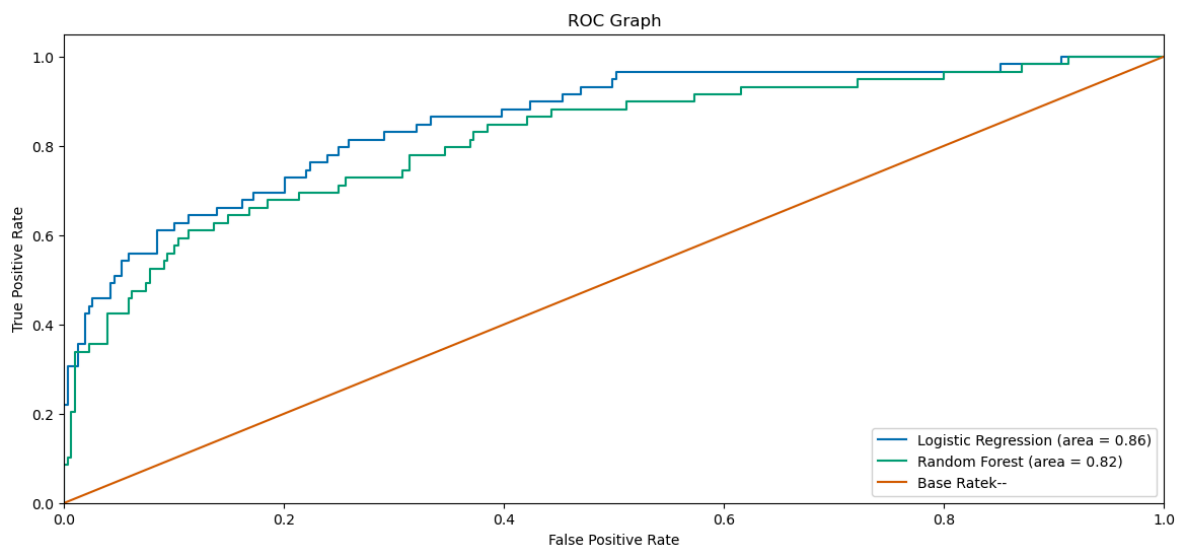
```
In [115]: rf_opt.fit(X_train, y_train) # fit optimised model to the training data
probs = rf_opt.predict_proba(X_test) # predict probabilities
probs = probs[:, 1] # we will only keep probabilities associated with the
rf_opt_roc_auc = roc_auc_score(y_test, probs) # calculate AUC score using test
print('AUC score: %.3f' % rf_opt_roc_auc)
```

AUC score: 0.816

```
In [116]: # Create ROC Graph
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, log_opt.predict_proba(X_test)[:,1])
rf_fpr, rf_tpr, rf_thresholds = roc_curve(y_test, rf_opt.predict_proba(X_test)[:,1])
plt.figure(figsize=(14, 6))

# Plot Logistic Regression ROC
plt.plot(fpr, tpr, label='Logistic Regression (area = %.2f)' % logit_roc_auc)
# Plot Random Forest ROC
plt.plot(rf_fpr, rf_tpr, label='Random Forest (area = %.2f)' % rf_opt_roc_auc)
# Plot Base Rate ROC
plt.plot([0,1], [0,1], label='Base Rate' 'k--')

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Graph')
plt.legend(loc="lower right")
plt.show()
```



In []:

